

LECTURE - 3

Monday
24/8/26.

□ Divide & Conquer:

- Divide: Break problem in sub-problems.
- Conquer: find solution of sub-prob. directly
- Combine: the solutions of subprobs. to solve larger prob.

⇒ Recursion: General & Base Case.

time func. → size of data/actual large prob.

$$T(n) = a T(n/b) + D(n) + C(n)$$

Total no. of subprobs.
(decides no. of
nodes in tree)

Size of
Sub prob.

Time to divide
a prob. into subprob.

Time to
Combine

⇒ Binary Search:

key = 23

1	2	3	4	5
5	8	11	13	20
l		m		h

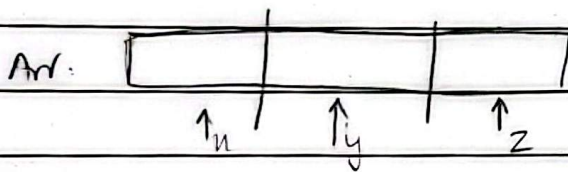

```

BinSearch ( Arr[L, h] ) {
    if (L > h)
        return
    else
        m = (L+h)/2
        if key == Arr[m]
            return true
        else if (key > Arr[m])
            BinSearch (Arr, m+1, h)
        else
            BinSearch (Arr, m-1, h)
}

```

* Since the subprob. is half of the array or half of data (i.e. n) $\therefore (n/b)$ in recurrence relation is ' $n/2$ '

* Since we're just taking care of one part of the problem i.e. half of array $\therefore a=1$.



If we'll taking care of three parts of array like x, y, z ; then $a=3$

Recursion Tree :

$T(n) \rightarrow T(n/2) \rightarrow T(n/4) \rightarrow T(n/8) \dots T(1)$

n	n	n	n	n	$\leftarrow$ Cost of division
$(n/2^0)$	$(n/2^1)$	$(n/2^2)$	$(n/2^3)$	1	$\leftarrow$ Size of sub prob.

* Since division cost same at each step $\therefore$ it'll be a constant n .

$\therefore D(n) = n$

* Since it's a searching problem $\therefore$ no combine cost $\therefore C(n) = 0$.

Height of tree : $(n/2^k) = 1$
 $n = 2^k \rightarrow \lg(n) = \lg(2^k)$

$\lg(n) = K \lg(2)$

height = $\lceil K = \lg(n) \rceil$

Costs 'C' to
divide at each step.

Date _____

Recurrence relation of Binary Search = $\log_2(n) * C$